



Урок 10 > Дополнительные подсистемы

> Ограничение нагрузки на систему

Важно предусмотреть механизм ограничения нагрузки, даже если система хорошо оптимизирована и развёрнута на хорошем железе.

Какие варианты ограничения нагрузки чаще всего применяют:

- Количество запросов от одного пользователя;
- Количество запросов от одного устройства;
- Количество запросов от одного IP-адреса.

Некоторые механизмы ограничений служат и целям безопасности:

- Ограничение количества неудачных попыток предотвращает атаку грубой силы (brute-force), когда злоумышленники хотят получить доступ к данным путём перебора паролей;
 - Ограничение RPS может предотвратить DDoS атаку.
-

Размещение модуля ограничения

- На клиентской части приложения. Вариант не надёжный: запросы к серверу можно отсылать не только из приложения, а еще из других специальных приложений, например, с Postman. Такие запросы пройдут мимо ограничителя;

- На серверной части — вариант хороший, но дополнительно нагружает сервер основного приложения;
 - Самым оптимальным вариантом остается вынести ограничитель в отдельный модуль между клиентом и сервером. Такой вариант снимает нагрузку с основного сервиса и одновременно служит буфером для любых запросов.
-

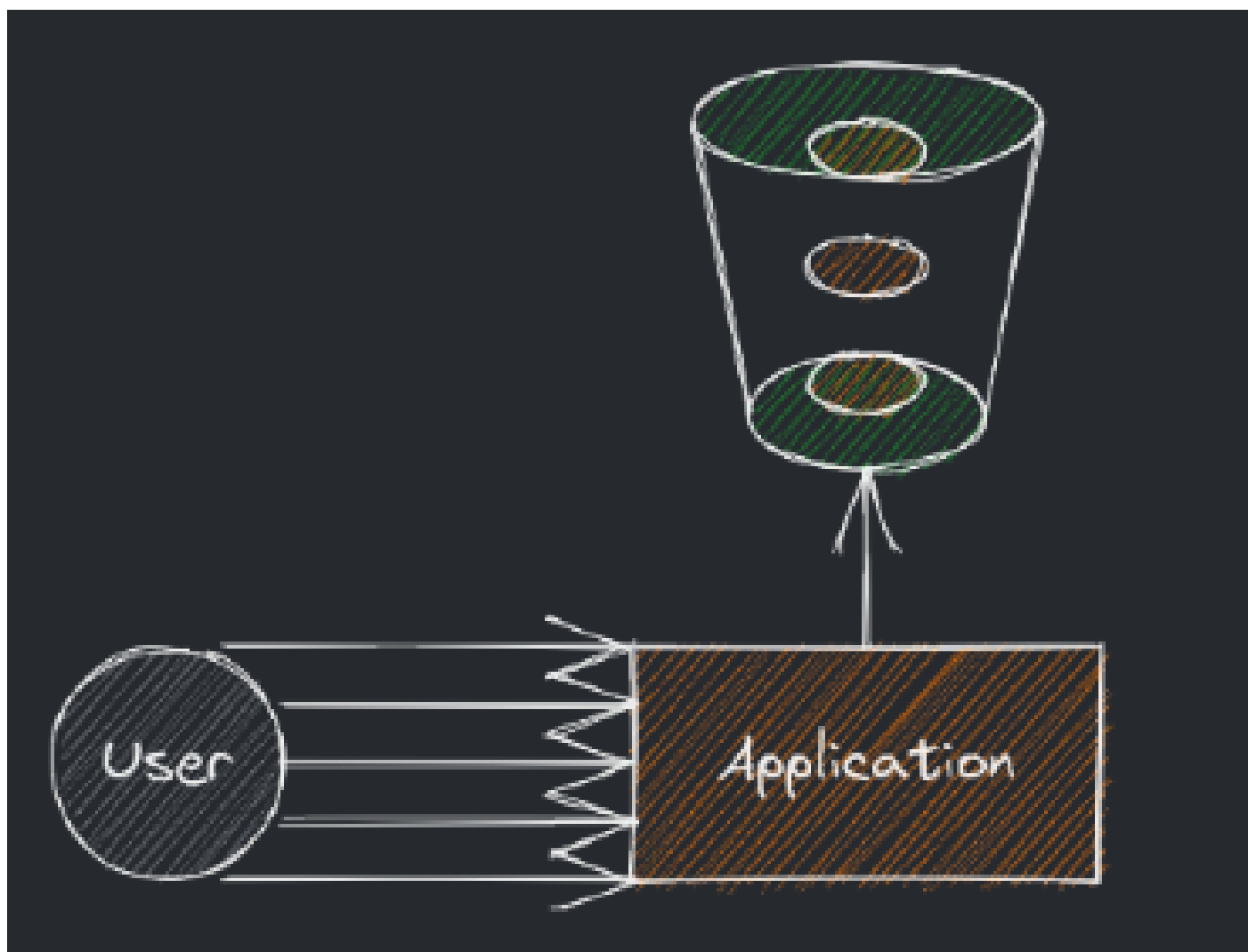
Требования к модулю ограничения нагрузки

- Возможность корректно ограничить запросы к конкретному API в количестве N за время T ;
- Ограничитель должен быть доступен распределённо и учитывать нагрузку с разных серверов или процессов;
- Ограничитель должен быть доступен всегда;
- Применение ограничителя не должно сильно сказываться на быстродействии.

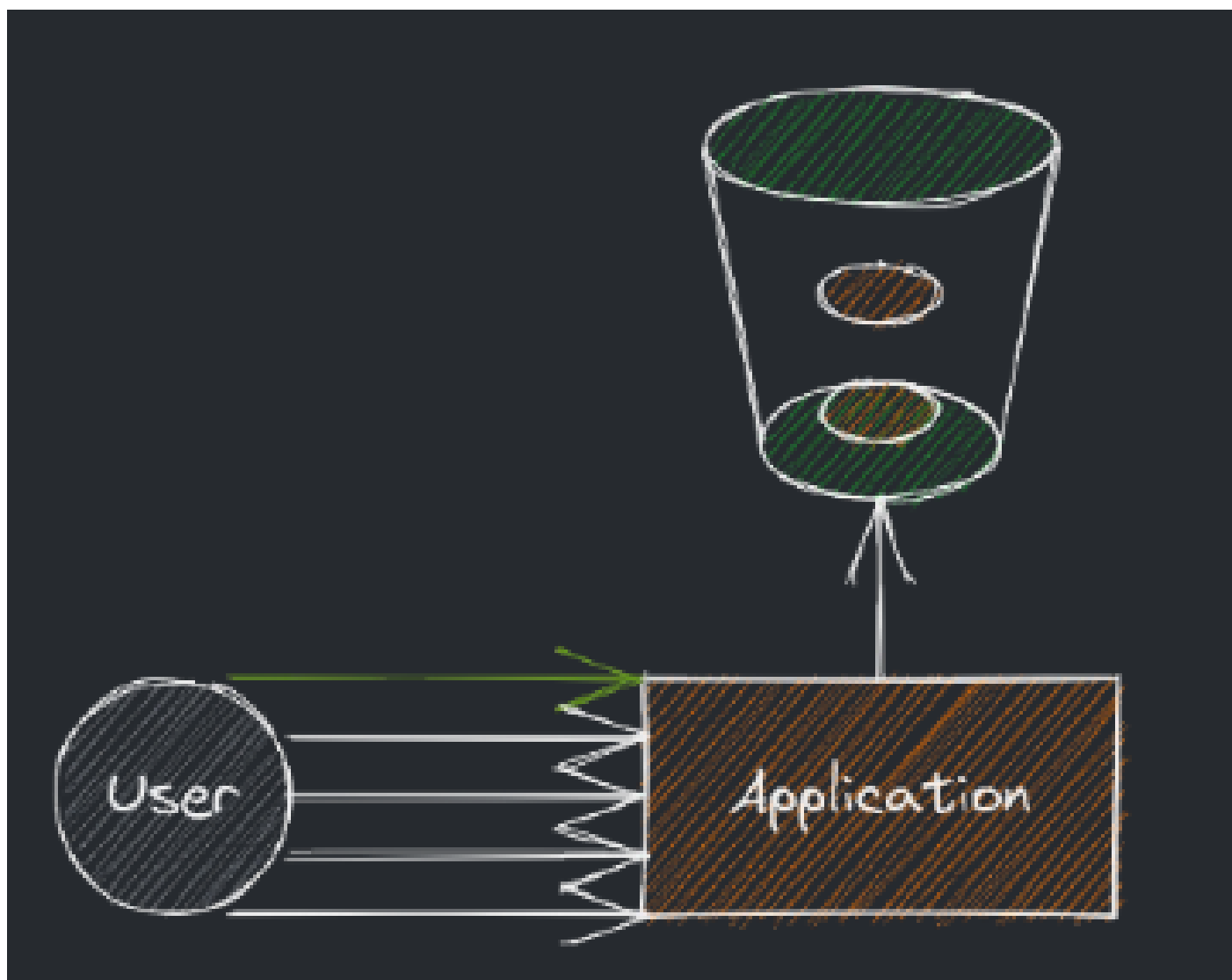
> Алгоритмы ограничения нагрузки

Алгоритм ведро с токенами

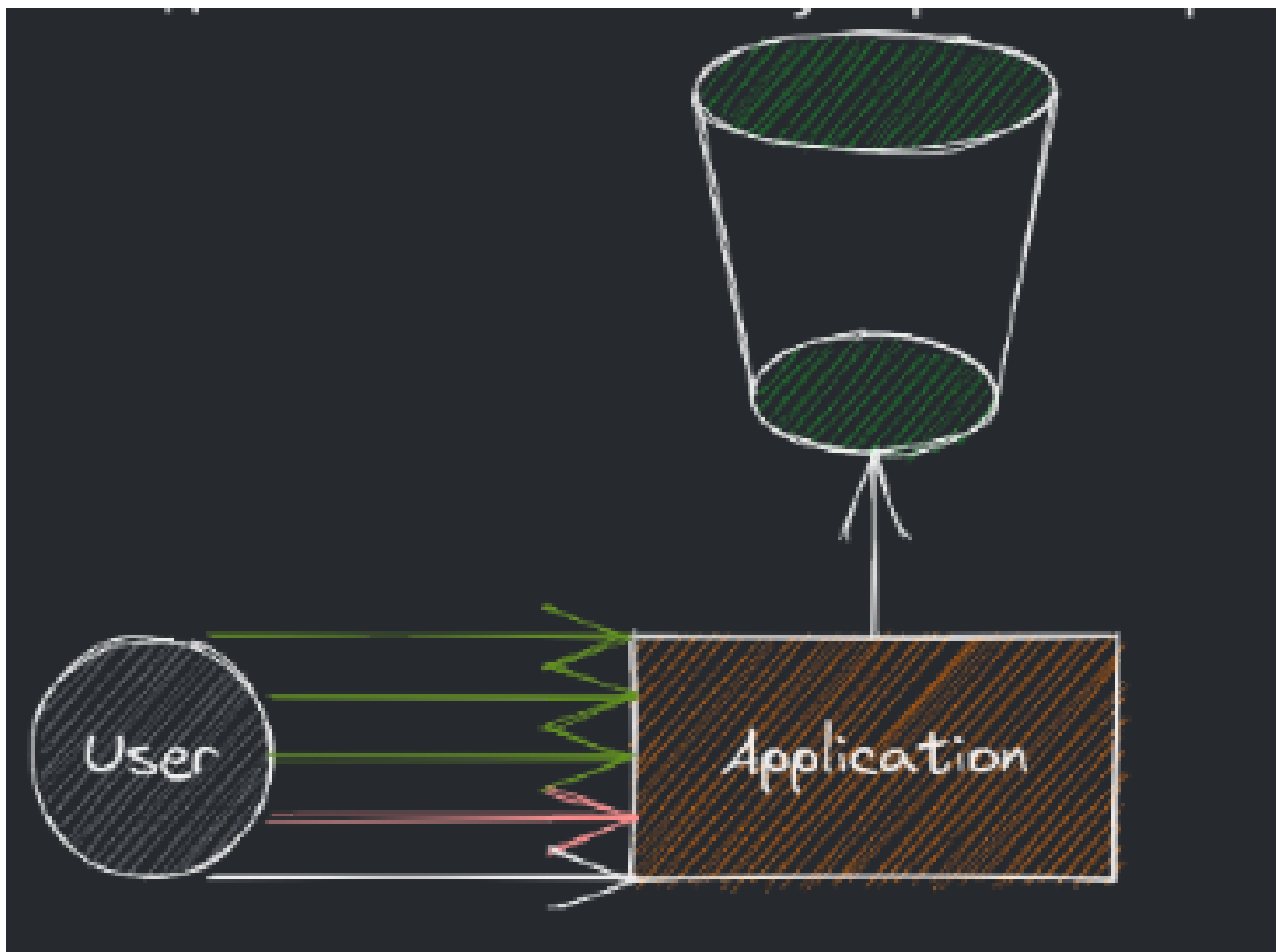
- Для контейнера с токенами используется стек;
- Изначально стек заполнен;
- Устанавливается периодичность добавления в стек токена;
- При каждом запросе от пользователя из стека удаляется один токен;
- При исчерпании токенов запросы от пользователя не принимаются до момента пополнения стека новым токеном.



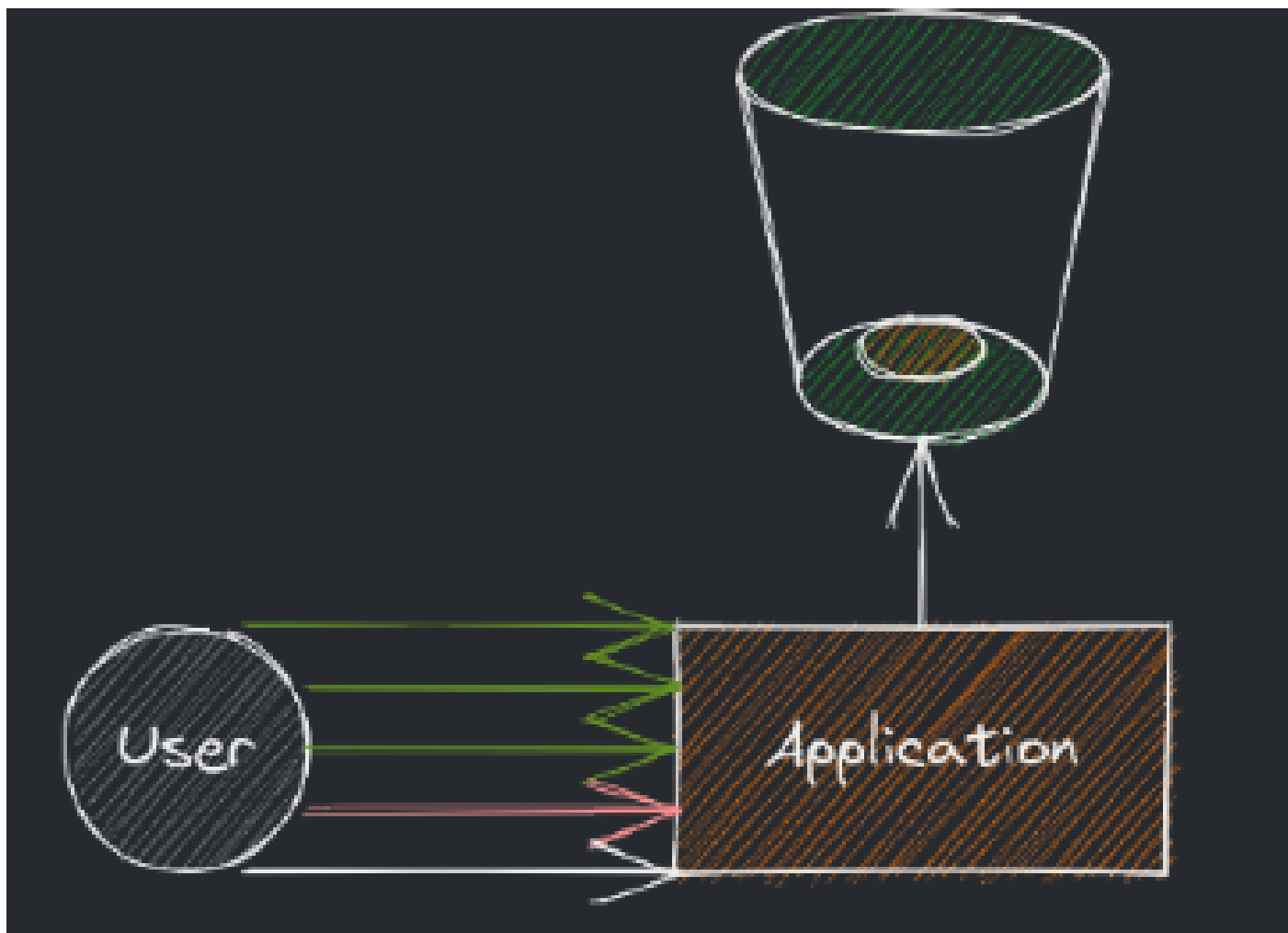
Стек заполнен



Пользователь сделал запрос и один токен сгорел



Когда свободные токены закончились, запрос блокируется

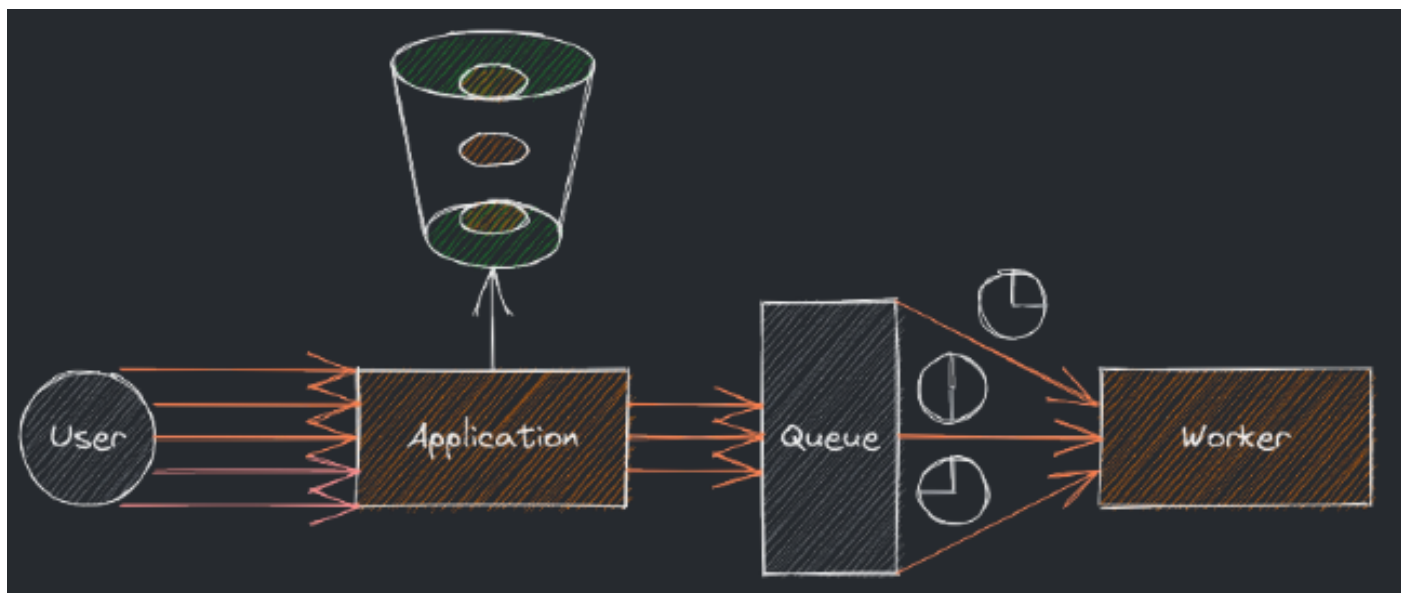


Через установленное время токены начинают восстанавливаться и пользователь сможет отправить запрос

К минусам алгоритма можно отнести тот факт, что если большое количество пользователей в конкретный момент времени начнут тратить свои токены, приложение получит максимальную нагрузку и может сломаться.

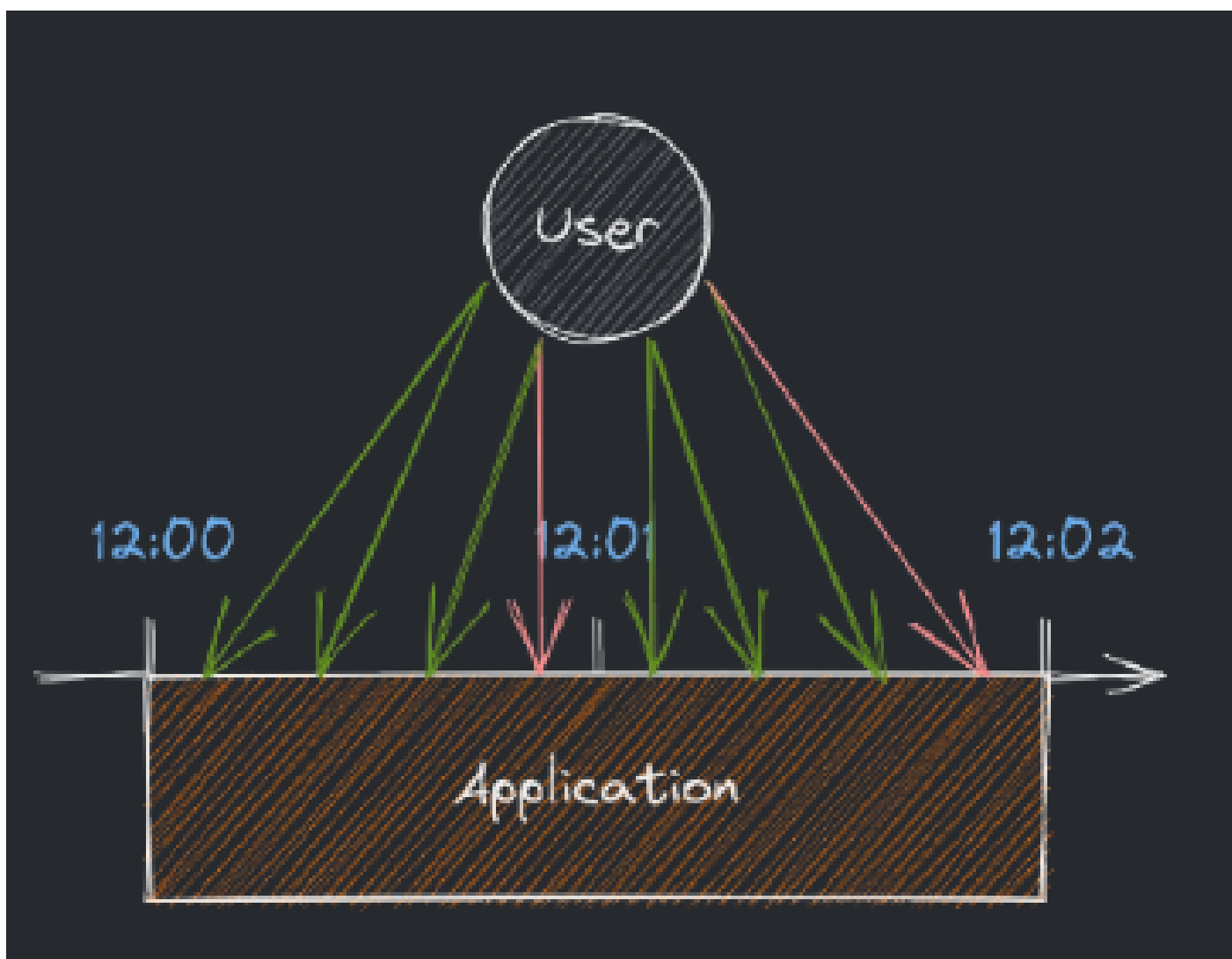
Алгоритм протекающего ведра

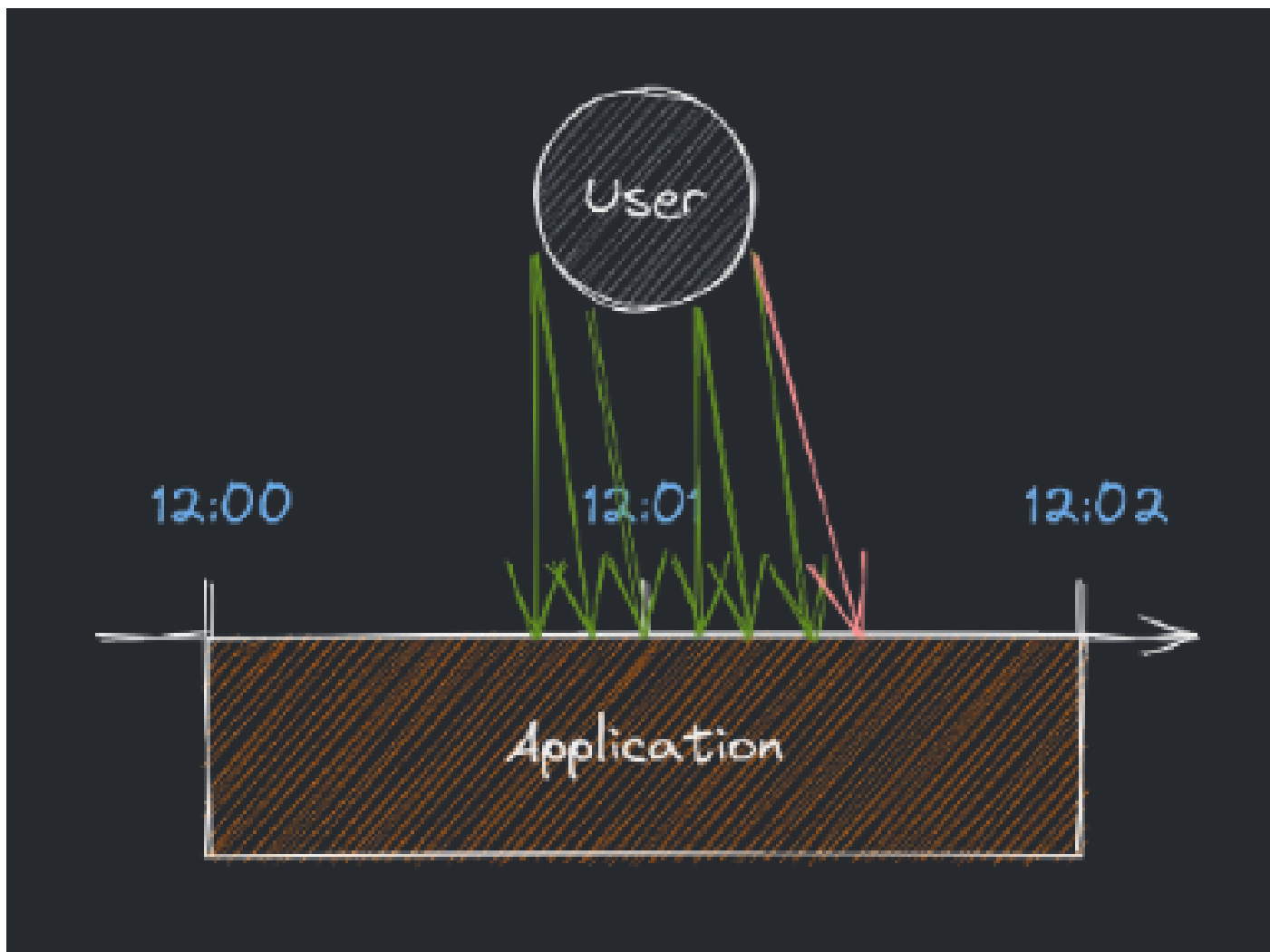
- В приложении также есть стек с токенами;
- Дополнительно после стека установлена очередь исполнения. Т.е. запросы, прошедшие через стек, исполняются не сразу, а устанавливаются в общую очередь и исполняются по мере возможности сервиса.



Алгоритм с фиксированным окном

Ограничиваем запросы в равные промежутки времени.

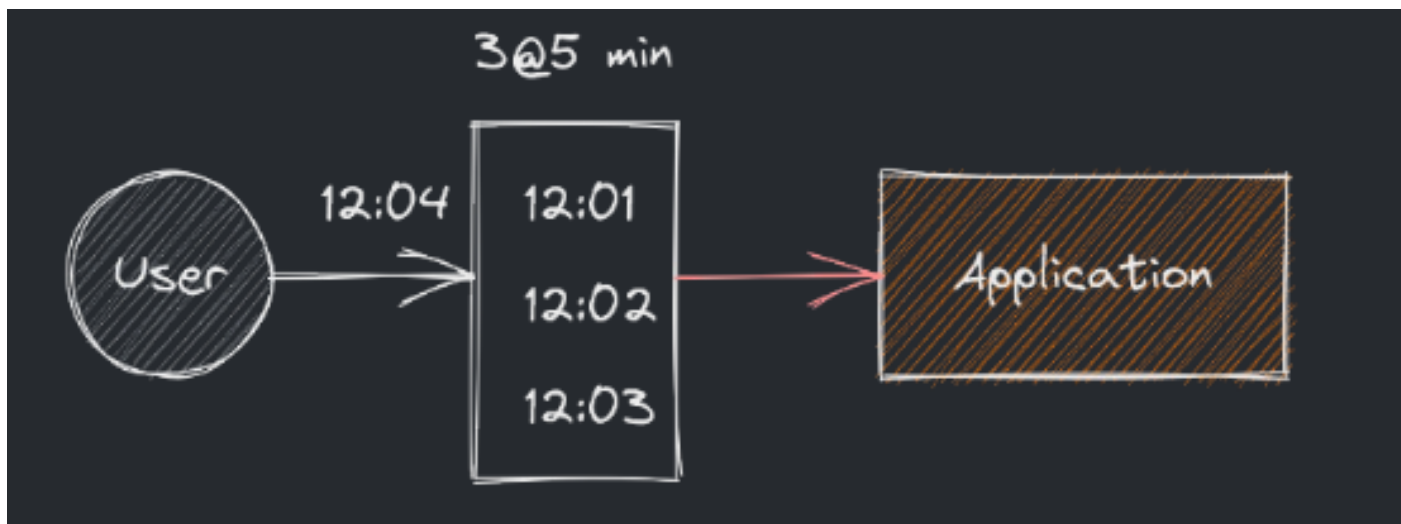




Всё-таки предельная нагрузка может достигаться, если все запросы придут на период между двумя промежутками времени

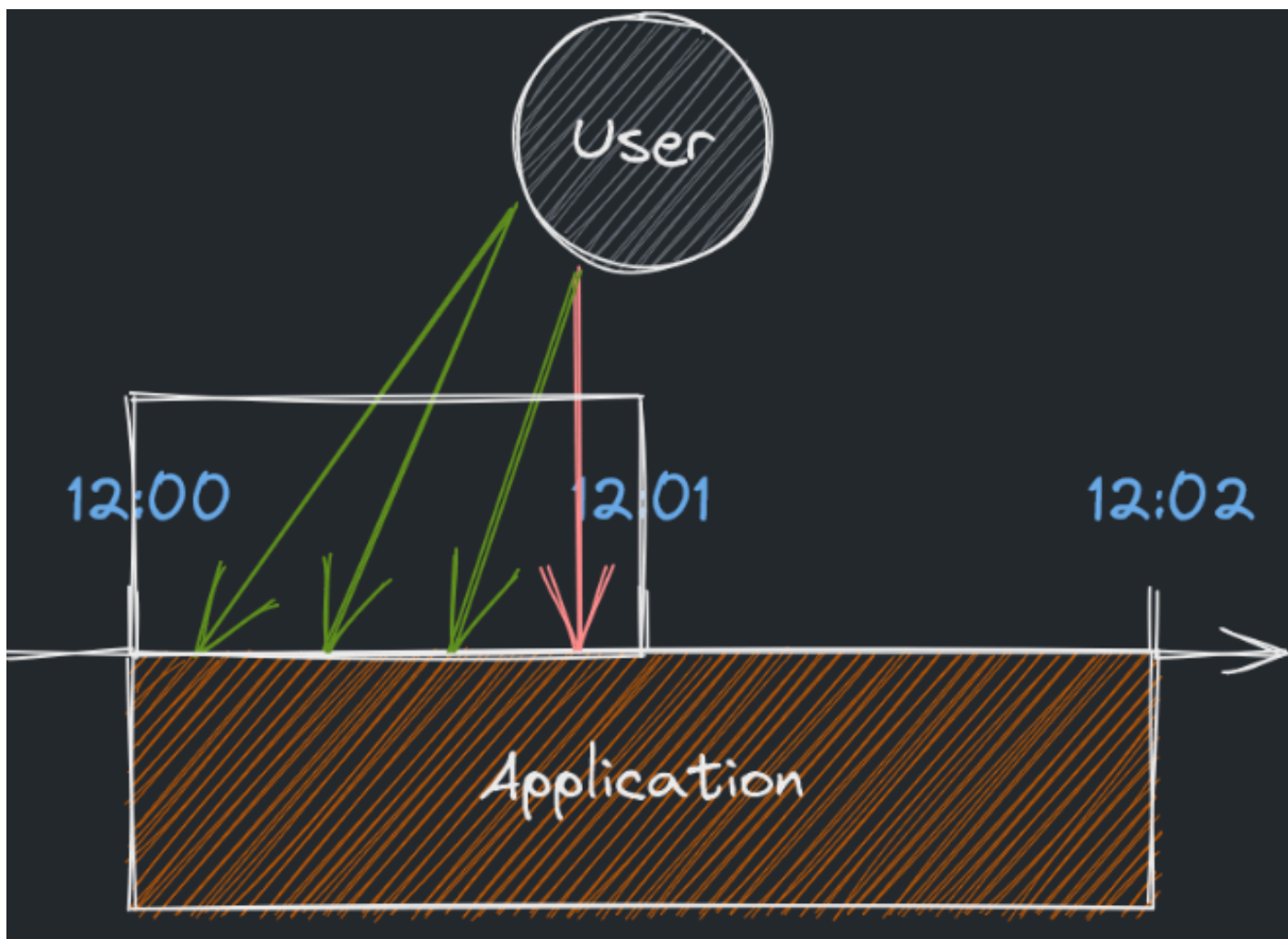
Алгоритм с обновленным логом

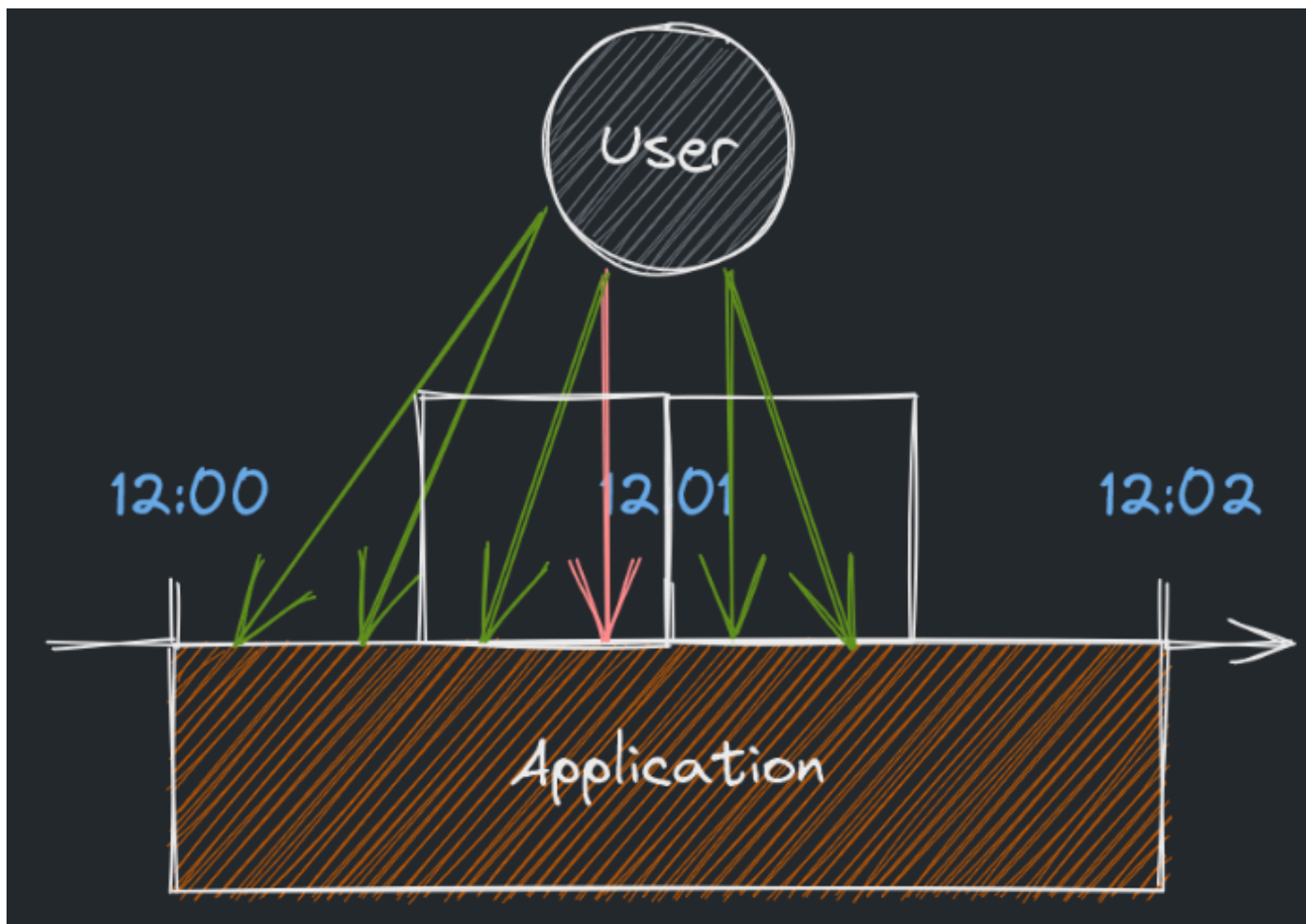
- Логируем каждый успешный запрос от пользователя;
- При поступлении нового запроса чистим старые, если такие есть;
- Если в логе нет места для текущего запроса, пропускаем его.



Алгоритм со скользящим окном

- Гибридный подход между фиксированным окном и обновлённым логом;
- Оценивает количество запросов во временном окне как взвешенную сумму статистик по периодам;
- В окне, содержащем 1/3 первой минуты и 2/3 второй, оцениваем запросы как $\frac{1}{3} * n1 + \frac{2}{3} * n2$.





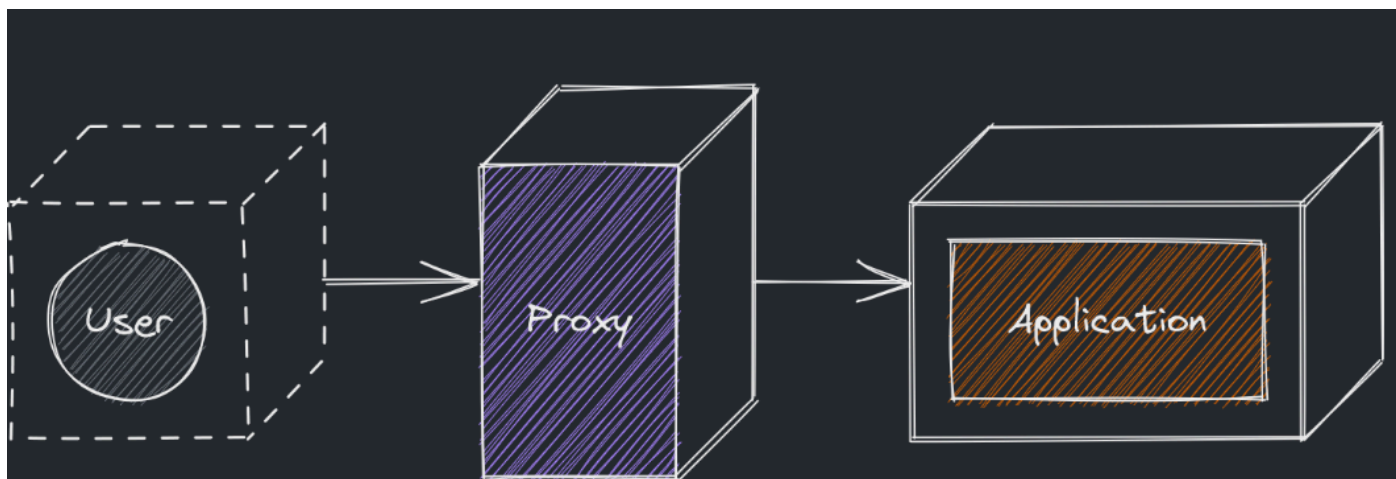
> Модули защиты системы

Способы защиты системы от злоумышленников

- Адреса реальных серверов скрывают за обратными прокси;
 - Файрвол для ограничения доступа извне.
-

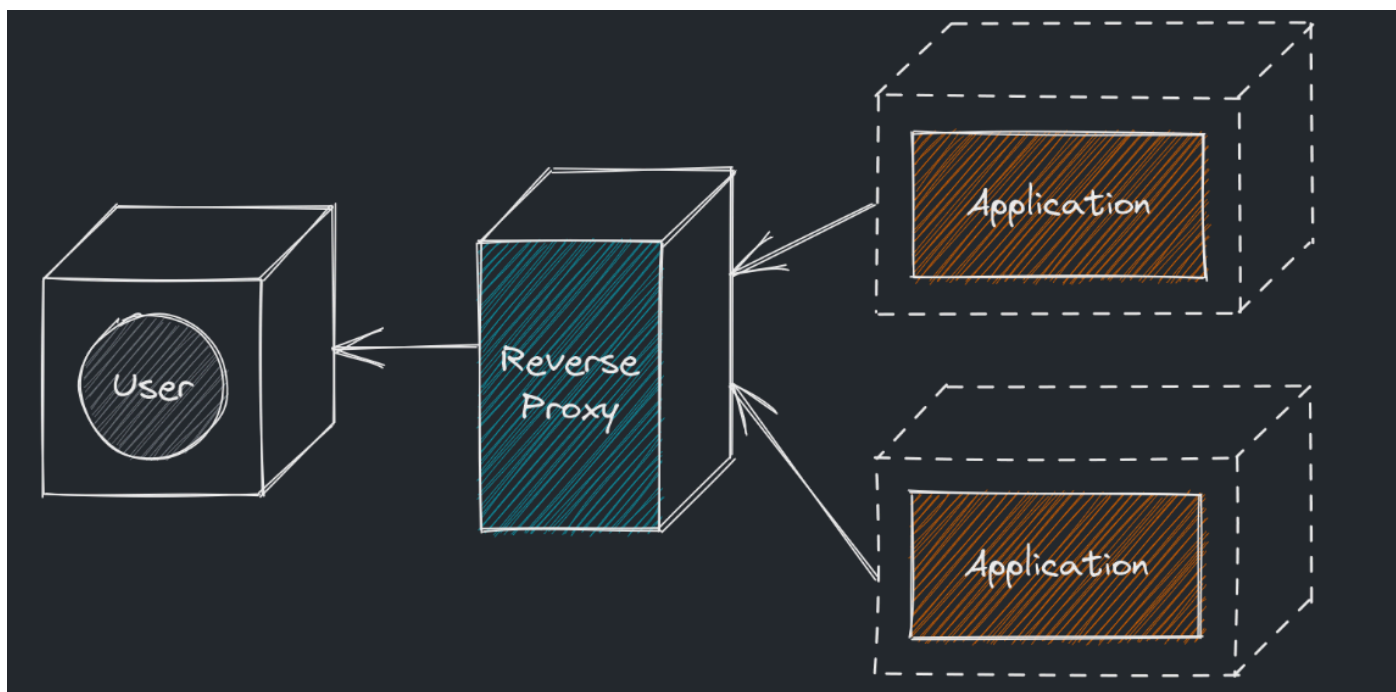
Работа прокси

Обычный прокси используют для сокрытия истинного источника обращения к приложению.



Сервис не видит реальный IP пользователя

Обратный прокси-сервер используется для сокрытия реальных серверов, отвечающих на запросы пользователя.

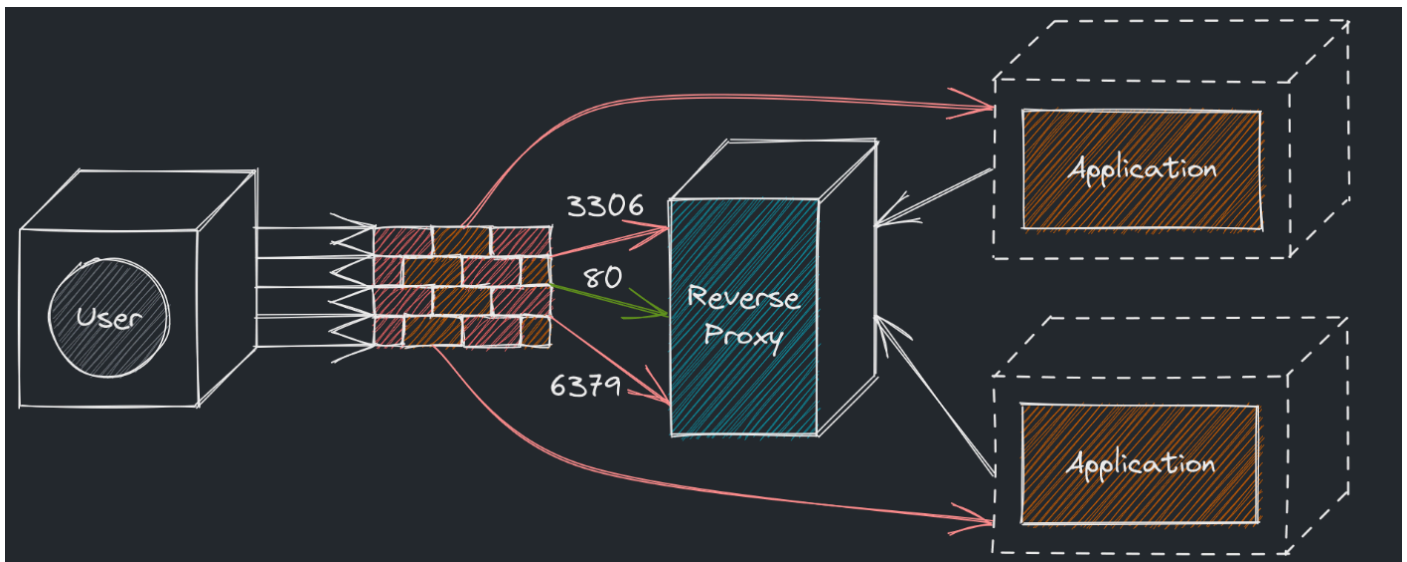


Таким образом, все запросы идут на IP прокси-сервера, при этом адреса настоящих серверов известны только прокси, который переадресует запрос к настоящему серверу, получает ответ и отправляет пользователю

При этом подходе существует вероятность «уронить» прокси-сервер, но реальные сервера останутся в рабочем состоянии.

Файрвол

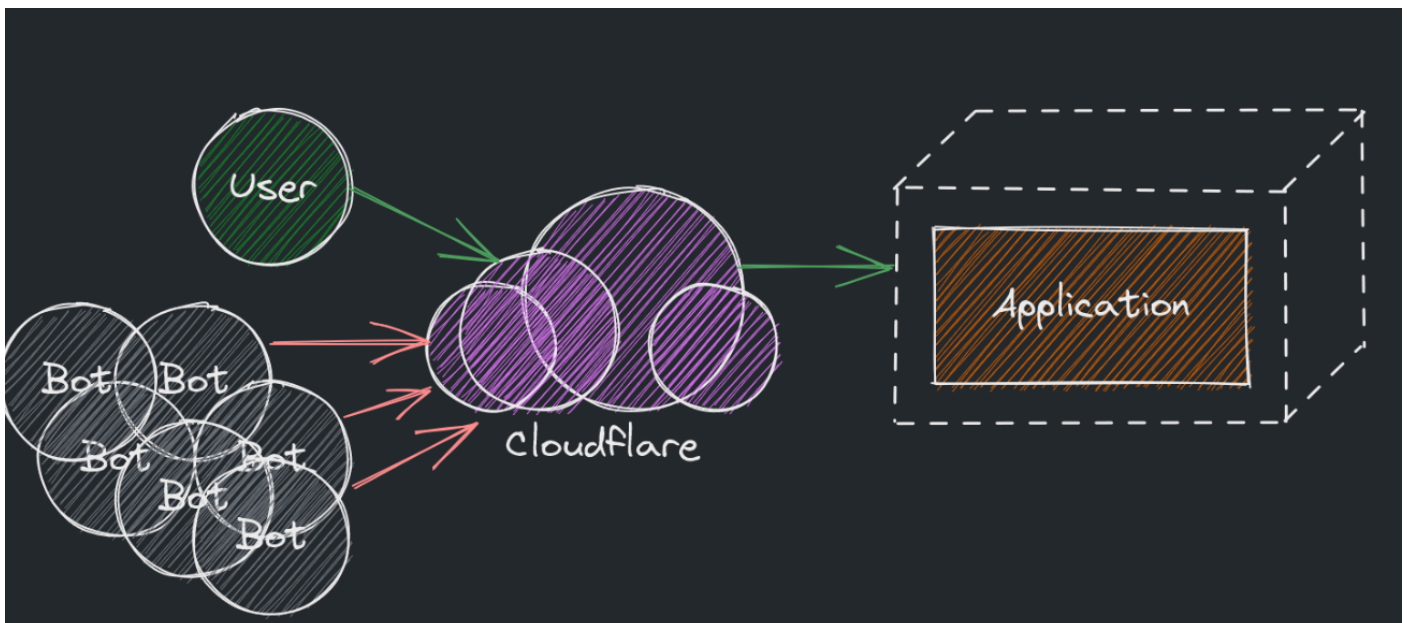
Файрволы ограничивают возможность обработки запросов. В модуле устанавливают конкретные IP-адреса, порты или конкретных пользователей, с которых можно обрабатывать запрос. Если запрос пришел с другого порта или адреса, он не пройдет к серверу.



Типы фаерволов:

- Пакетные фильтры — самые простые, сверяют IP, порт, протокол и другие базовые свойства запроса;
- Сеансового уровня — соединяются от имени внутренних сервисов, пропускают пакеты только для них;
- Инспекторы состояний — контролируют пакеты, сессии и приложения на основе внутренних таблиц разрешения.

Есть возможность использовать услуги внешних сервисов, например [Cloudflare](https://www.cloudflare.com/).

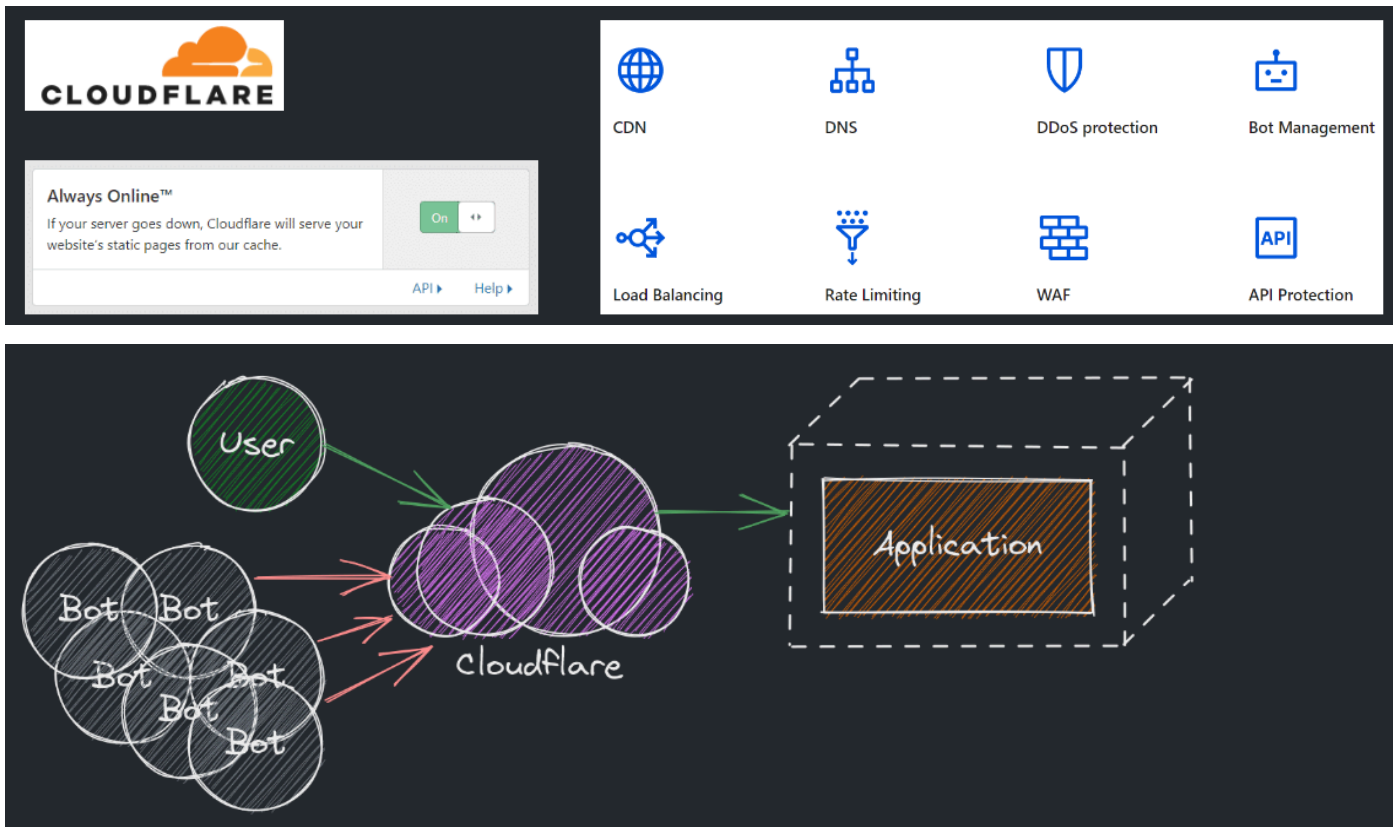


> Внешние сервисы

При необоснованности выделения ресурсов можно прибегнуть к услугам внешних сервисов.

Компании (например, Cloudflare) предоставляют комплексные услуги, включающие файрвол как сервис, защиту от целенаправленных DDoS атак, балансировщик и ограничитель нагрузки, CDN хранилище и другие.

Они также позволяют отобразить кешированную версию сайта на случай падения основного приложения.



При необоснованности выделения ресурсов можно прибегнуть к услугам внешних сервисов

> Подсистемы мониторинга

Кроме балансировщиков нагрузки имеет смысл и самим отслеживать критичные показатели системы:

- Распределение времени ответа тех или иных подсервисов;
- Распределение положительных ответов и различных ошибок.

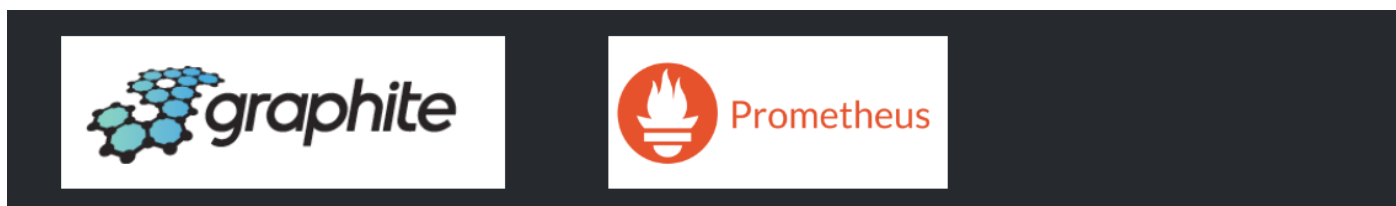
В случае наличия серьёзных отклонений можно уведомить инженеров или администраторов.

Все показатели стоит сохранять в течении какого-то времени для ретроспективного анализа.

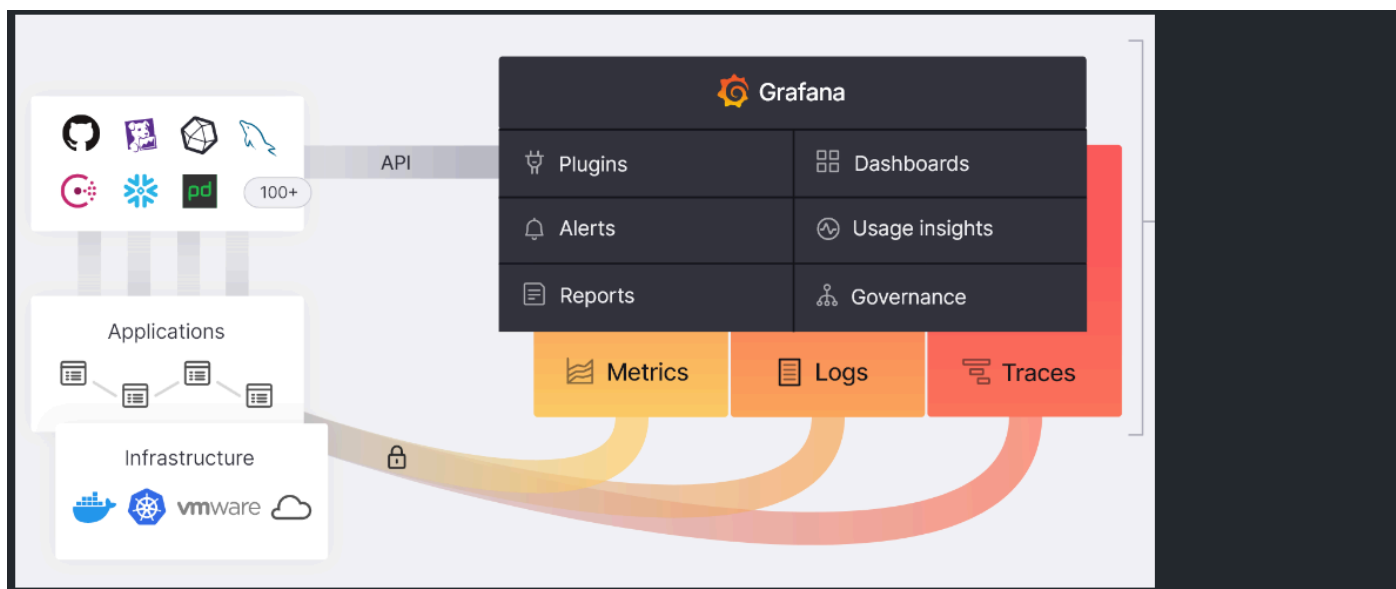
Основные парадигмы отслеживания показателей

1. Самостоятельное логгирование метрик приложения — [Graphite](#)

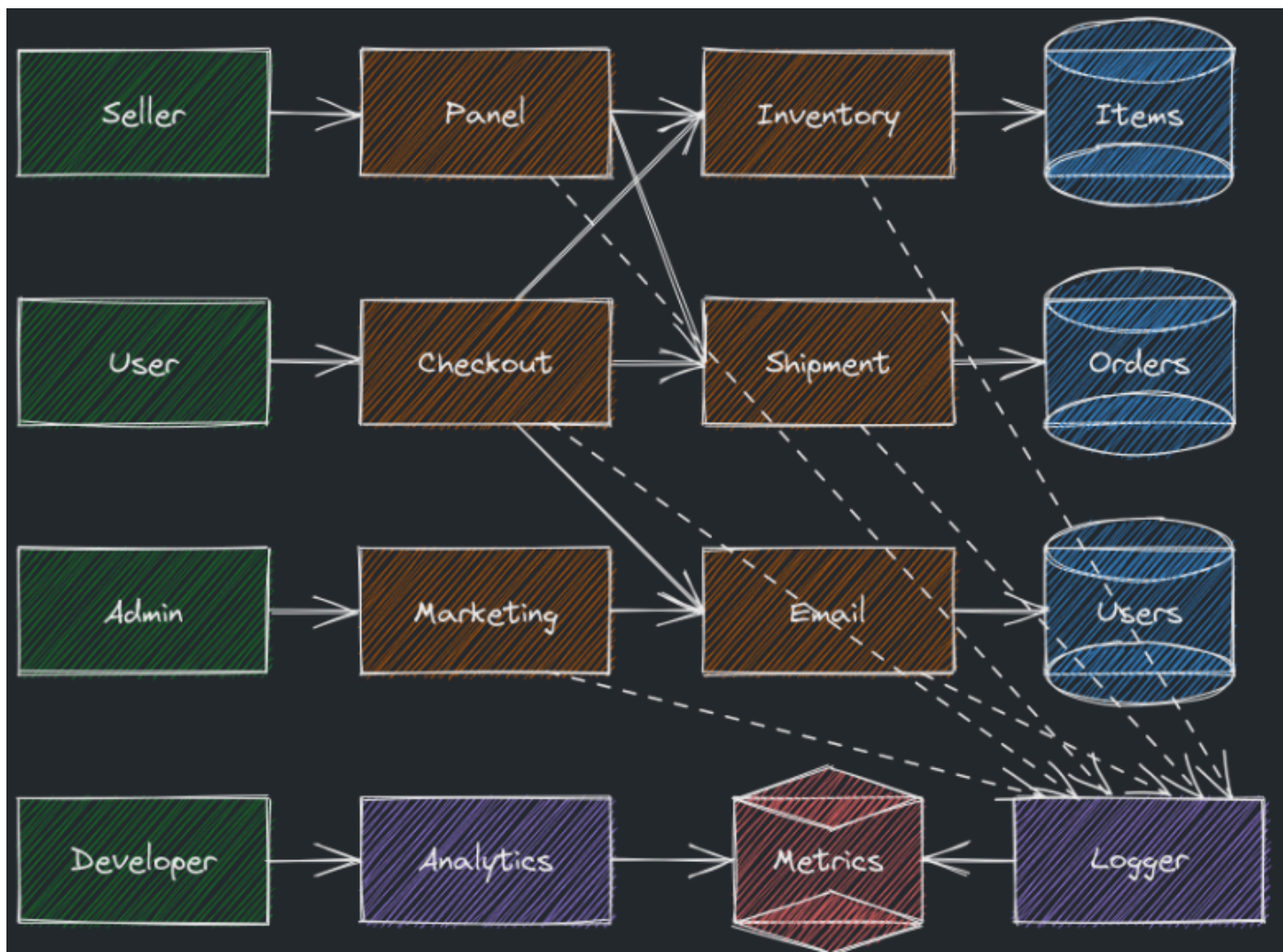
2. Сбор метрик внешними мониторами через API приложения — [Prometheus](#)



[Grafana](#) — система с удобным интерфейсом для мониторинга показателей. Сбор самих метрик можно осуществлять силами Graphite или Prometheus.



Для сбора метрик в большом приложении можно организовать подсистему логирования с каждого из сервисов.



> Внешние подсистемы

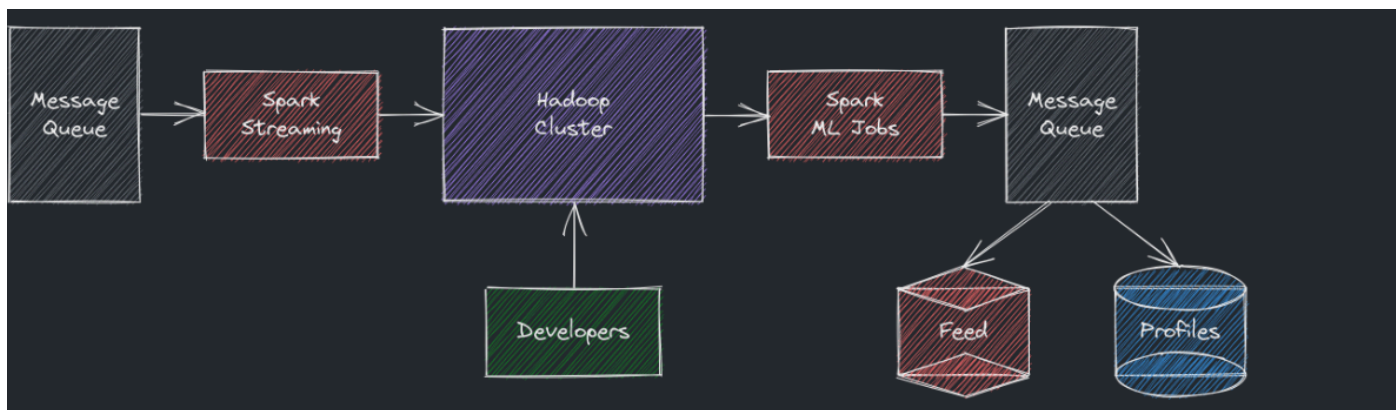
При реализации новой системы внутри компании не стоит забывать и о других системах. Они могут быть уже в работе или только гипотетически рассматриваться для реализации.

Для условной системы логирования просмотров или кликов пользователя ваши коллеги могут:

- проводить различные А/Б тесты по привлекательности той или иной страницы;
- заниматься офлайн сегментацией пользователя по интересам;
- планировать перейти на Hadoop для обучения сложных моделей и их внедрения.

Так может выглядеть взаимодействие с командой ML инженеров:

- сырые данные предобрабатываются и передаются на хранение в MLкластер;
- с применением ресурсов MLкластера по расписанию обучаются новые модели;
- также разработчики проводят эксперименты по улучшению моделей и обучению новых;
- задания периодически обрабатывают данные и передают результаты в нашу систему.



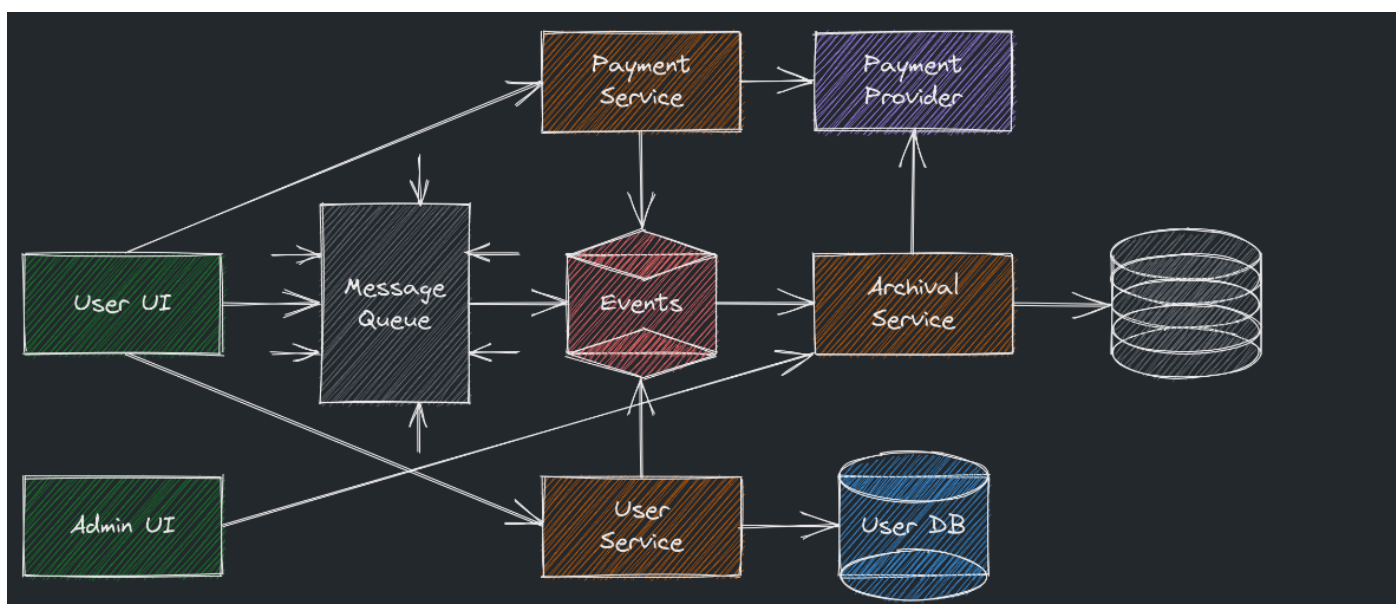
> Внешние проблемы

Если вы храните пользовательский контент или взаимодействуете с чувствительными данными, а тем более с ресурсами, оплаченными пользователем, так или иначе могут возникнуть претензии.

Как возможно справиться с такими ситуациями?

- Ранее уже обсуждали критичность ACID-требований для чувствительных транзакций;
- Сам процесс оплаты всегда можно возложить на внешние сервисы оплаты типа Stripe;
- Логировать все взаимодействия стоит не только для аналитики, но и для безопасности.

Так при исследовании проблем будет возможность восстановить историю или обратиться к провайдеру.



Так может выглядеть часть системы, максимально подготовленной к разбору полётов

> Выводы

— Если нагрузка превосходит ожидания, можно задуматься о её ограничении

- Если атаки на систему не пережить самим, для защиты можно использовать внешние сервисы
- Уже со старта необходимо позаботиться о мониторинге всех составляющих системы
- Не стоит забывать и о других командах и их системах, продумайте взаимодействие с ними
- При работе с пользовательским контентом надо иметь возможность изучить историю изменений